Dhaka International University

Department of Computer Science and Engineering

Project Report

Database Management System Lab

Dental Clinic Management System

An Oracle SQL Based Database Project

Course Code	0612-304
Course Teacher	Mobarak Hossain Al-Mamon Lecturer, DIU

Submitted By

Name	Md Sanjid Islam
Reg No	CS-D-98-23-127242
Batch	D-98
Roll	21
Department	Computer Science and Engineering

Date of Submission: 11 November 2026

Table of Contents

Chapter	Title	Page
1	Introduction	3
2	Project Overview	3
3	ER Model and ER Diagram	4
4	ER-to-Relational Mapping and Table Description	5
5	DDL - Table Management	6
6	DML - Data Manipulation	7
7	Sample Queries and Output	8
8	Conclusion	12

List of Tables: Table 4.1 gives the description of the six tables used in this project. The sample data used in the queries of Chapter 7 is inserted in Chapter 6.

1. Introduction

A dental clinic keeps records of its patients, doctors, appointments and payments. In many small clinics these records are still kept in paper registers. Searching an old record from a register takes time, the same information is written again and again, and there is no easy way to find out how many patients a doctor has seen or how much money is still due from a patient.

In this project, a small database has been designed for a dental clinic. The database is created and managed only with SQL queries in Oracle. There is no website or software interface in this project. All the work - creating tables, inserting records and searching information - is done by writing queries in the terminal.

The main objective of this project is to apply the topics taught in the DBMS lab course, such as table creation, data manipulation, filtering, sorting, aggregate functions, grouping and subqueries, on a real-life type of problem. A secondary objective is to design the database properly, starting from an ER model and then converting it into relational tables.

2. Project Overview

The Dental Clinic Management System stores the daily information of a clinic that has more than one chamber. A patient books an appointment with a doctor for a particular treatment. After the treatment, the payment for that appointment is recorded. The system can answer questions such as which patients are from a certain area, which doctor has the highest consultation fee, how much money has been collected in total, and which appointments are still unpaid.

2.1 Tables used in the project

The database contains six tables:

- **Chamber** - information about the clinic chambers.
- **Doctor** - information about the doctors and the chamber they sit in.
- **Patient** - information about the patients.
- **Treatment** - the treatments offered by the clinic and their fees.
- **Appointment** - which patient visits which doctor for which treatment, and on which date.
- **Payment** - the payment made against an appointment.

2.2 Tools used

Oracle Database is used as the DBMS, and the queries are written in SQL. The project is run in a terminal-based environment such as Oracle SQL*Plus or Oracle Live SQL.

3. ER Model and ER Diagram

An entity is an object about which data is stored, such as a patient or a doctor. Each entity has attributes, which are its properties. In this project there are six entities: Chamber, Doctor, Patient, Treatment, Appointment and Payment.

Patient_ID is a key attribute of Patient. The address of a patient is a composite attribute, as it is divided into Area and City. The age of a patient is a derived attribute, because it can be calculated from the date of birth. Doctor is divided by specialization, such as General Dentist, Oral Surgeon and Orthodontist; this is an example of specialization.

Chamber, Doctor, Patient, Treatment and Appointment are strong entities, because each of them has its own primary key. Payment is a weak entity. A payment cannot exist without an appointment, and it has no single key of its own, so it is identified by Appointment_ID together with Payment_No.

One chamber has many doctors, so the relationship between Chamber and Doctor is one-to-many. A patient may have many appointments, and a doctor may attend many appointments. In this way, Appointment removes the many-to-many relationship between Patient and Doctor. An appointment cannot exist without a patient and a doctor, so its participation is total, while payment is partial, because an appointment may remain unpaid.

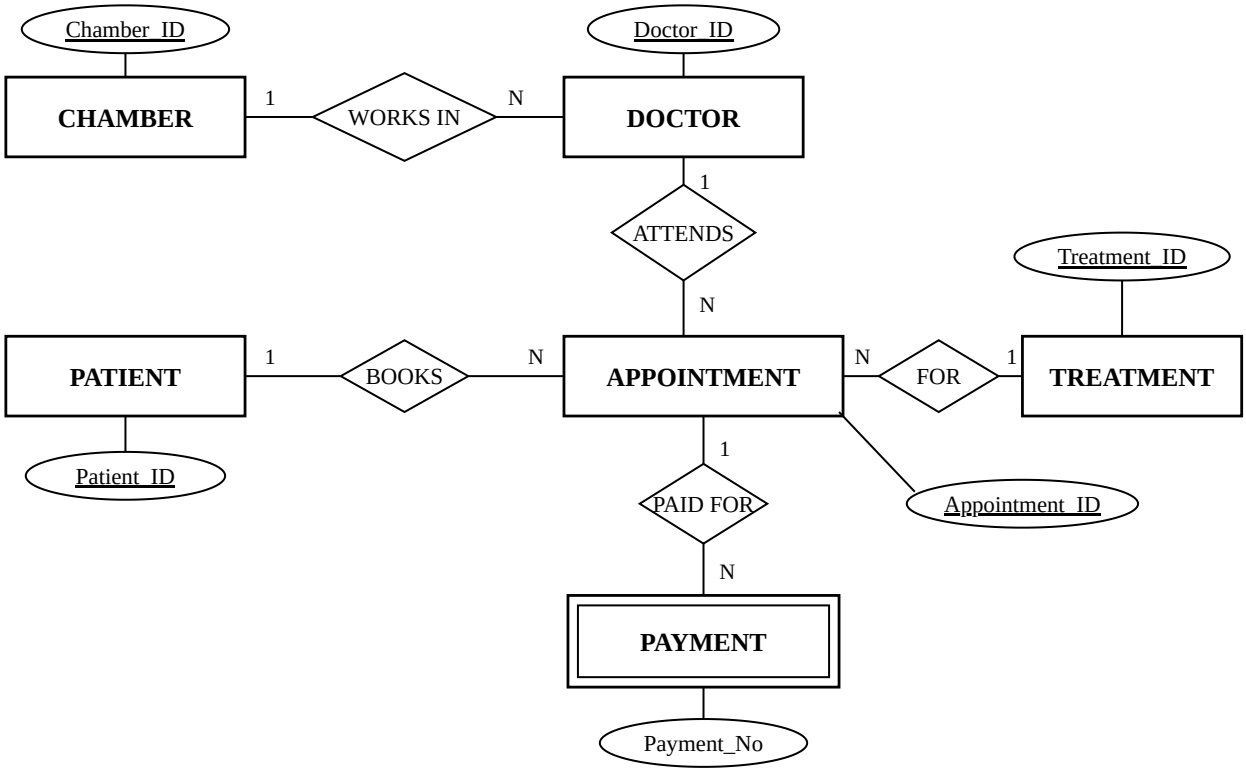


Figure 3.1: ER Diagram of the Dental Clinic Management System. Rectangles are entities, diamonds are relationships, ovals are key attributes, and the double rectangle (Payment) is the weak entity.

4. ER-to-Relational Mapping and Table Description

The ER model is converted into relational tables by following the normal mapping rules. Each strong entity becomes a table of its own, and its key attribute becomes the primary key of that table. For a one-to-many relationship, the primary key of the table on the "one" side is kept as a foreign key in the table on the "many" side. For example, Chamber_ID is kept as a foreign key in the Doctor table. The weak entity Payment becomes a table whose primary key is made of the primary key of its owner entity, Appointment, together with its own partial key, Payment_No.

4.1 Relational schema

```
CHAMBER    (Chamber_ID, Chamber_Name, Area, City, Phone)
DOCTOR     (Doctor_ID, Doctor_Name, Specialization, Qualification,
            Gender, Phone, Consultation_Fee, Chamber_ID)
PATIENT    (Patient_ID, Patient_Name, Date_of_Birth, Gender, Phone,
            Area, City, Email)
TREATMENT  (Treatment_ID, Treatment_Name, Category, Base_Fee,
            Duration_Min)
APPOINTMENT(Appointment_ID, Patient_ID, Doctor_ID, Treatment_ID,
            Appointment_Date, Status)
PAYMENT    (Appointment_ID, Payment_No, Payment_Date, Amount_Paid,
            Discount, Payment_Method)
```

4.2 Description of the tables

Table 4.1: Description of the six tables

Table	Primary Key	Foreign Key and purpose
Chamber	Chamber_ID	- Stores the name, area and phone of each chamber.
Doctor	Doctor_ID	Chamber_ID refers to Chamber. Stores the doctor's specialization and consultation fee.
Patient	Patient_ID	- Stores the patient's name, date of birth, phone and address (Area, City).
Treatment	Treatment_ID	- Stores the treatment name, category, base fee and duration.
Appointment	Appointment_ID	Patient_ID, Doctor_ID and Treatment_ID refer to Patient, Doctor and Treatment. Stores the date and status of a visit.
Payment	Appointment_ID + Payment_No	Appointment_ID refers to Appointment. Stores the amount paid, discount and payment method.

The structure of the tables given above is called the schema of the database. The rows stored in the tables at any particular time are called the instance of the database.

5. DDL - Table Management

Data Definition Language (DDL) commands are used to create and change the structure of tables. The data types NUMBER, VARCHAR2, CHAR and DATE are used here. PRIMARY KEY, NOT NULL and CHECK constraints are used so that wrong data cannot be entered. The tables are created as follows:

```
CREATE TABLE Chamber (
  Chamber_ID NUMBER(4) PRIMARY KEY,  Chamber_Name VARCHAR2(50) NOT NULL,
  Area VARCHAR2(30) NOT NULL,  City VARCHAR2(30) DEFAULT 'Dhaka',  Phone VARCHAR2(15));

CREATE TABLE Doctor (
  Doctor_ID NUMBER(4) PRIMARY KEY,   Doctor_Name VARCHAR2(60) NOT NULL,
  Specialization VARCHAR2(40) NOT NULL, Qualification VARCHAR2(60),
  Gender CHAR(1) CHECK (Gender IN ('M','F')),  Phone VARCHAR2(15),
  Consultation_Fee NUMBER(8,2) CHECK (Consultation_Fee > 0),  Chamber_ID NUMBER(4),
  CONSTRAINT fk_doctor_chamber FOREIGN KEY (Chamber_ID)
      REFERENCES Chamber(Chamber_ID));

CREATE TABLE Patient (
  Patient_ID NUMBER(5) PRIMARY KEY,  Patient_Name VARCHAR2(60) NOT NULL,
  Date_of_Birth DATE,  Gender CHAR(1) CHECK (Gender IN ('M','F','O')),
  Phone VARCHAR2(15) NOT NULL,  Area VARCHAR2(30),  City VARCHAR2(30) DEFAULT 'Dhaka');

CREATE TABLE Treatment (
  Treatment_ID NUMBER(4) PRIMARY KEY, Treatment_Name VARCHAR2(50) NOT NULL,
  Category VARCHAR2(30),  Base_Fee NUMBER(8,2) CHECK (Base_Fee > 0),
  Duration_Min NUMBER(3) CHECK (Duration_Min > 0));

CREATE TABLE Appointment (
  Appointment_ID NUMBER(6) PRIMARY KEY,  Patient_ID NUMBER(5) NOT NULL,
  Doctor_ID NUMBER(4) NOT NULL,  Treatment_ID NUMBER(4) NOT NULL,
  Appointment_Date DATE NOT NULL,
  Status VARCHAR2(12) DEFAULT 'Scheduled'
      CHECK (Status IN ('Scheduled','Completed','Cancelled')),
  FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),
  FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID),
  FOREIGN KEY (Treatment_ID) REFERENCES Treatment(Treatment_ID));

CREATE TABLE Payment (
  Appointment_ID NUMBER(6) NOT NULL,  Payment_No NUMBER(2) NOT NULL,
  Payment_Date DATE,  Amount_Paid NUMBER(8,2) CHECK (Amount_Paid >= 0),
  Discount NUMBER(8,2) DEFAULT 0,  Payment_Method VARCHAR2(15)
      CHECK (Payment_Method IN ('Cash','bKash','Card')),
  PRIMARY KEY (Appointment_ID, Payment_No),
  FOREIGN KEY (Appointment_ID) REFERENCES Appointment(Appointment_ID));
```

5.1 DESCRIBE and ALTER TABLE

The structure of a table can be seen with DESCRIBE, for example *DESCRIBE Patient;* The structure can be changed with ALTER TABLE:

```
ALTER TABLE Patient ADD (Email VARCHAR2(60));
ALTER TABLE Patient ADD (Temp_Note VARCHAR2(20));  ALTER TABLE Patient MODIFY (Phone
VARCHAR2(20));
ALTER TABLE Patient RENAME COLUMN Temp_Note TO Visit_Note;
ALTER TABLE Patient DROP COLUMN Visit_Note;
```

Here ADD adds a new column, MODIFY changes the size of a column, RENAME COLUMN changes a column name, and DROP COLUMN removes a column. TRUNCATE TABLE removes all rows but keeps the table, and DROP TABLE removes the table completely.

6. DML - Data Manipulation

Data Manipulation Language (DML) commands work with the data stored in the tables. Records are added with INSERT, changed with UPDATE and removed with DELETE. In Oracle, a date written as text is converted into a DATE value with TO_DATE. Some sample records are given below; the full data is inserted in the same way.

```
INSERT INTO Chamber VALUES (1, 'Laser Dental Care', 'Uttara', 'Dhaka', '01711940185');
INSERT INTO Chamber VALUES (2, 'Smile Care Dental', 'Mirpur', 'Dhaka', '01811223344');
INSERT INTO Chamber VALUES (3, 'City Dental Hospital', 'Dhanmondi', 'Dhaka', '01922334455');

INSERT INTO Doctor VALUES (101, 'Dr. Saima Akter', 'General Dentist', 'BDS, MDS', 'F',
'01711000001', 800, 1);
INSERT INTO Doctor VALUES (102, 'Dr. Shakhawat Hossain', 'Oral Surgeon', 'BDS, FCPS', 'M',
'01711000002', 1200, 1);

INSERT INTO Patient VALUES (1001, 'Rahim Uddin', TO_DATE('12-03-1995','DD-MM-YYYY'), 'M',
'01720111222', 'Uttara', 'Dhaka', 'rahim@mail.com');
INSERT INTO Patient VALUES (1003, 'Karim Islam', TO_DATE('05-11-1988','DD-MM-YYYY'), 'M',
'01920555666', 'Dhanmondi', 'Dhaka', NULL);

INSERT INTO Treatment VALUES (202, 'Root Canal', 'Surgery', 8000, 60);
INSERT INTO Appointment VALUES (5001, 1001, 101, 201, TO_DATE('02-11-2026','DD-MM-YYYY'),
'Completed');
INSERT INTO Payment VALUES (5001, 1, TO_DATE('02-11-2026','DD-MM-YYYY'), 2300, 0, 'Cash');
```

In total, 3 chambers, 5 doctors, 8 patients, 6 treatments, 12 appointments and 6 payments are inserted. The parent tables are filled first, because a child table cannot refer to a row that does not exist.

6.1 UPDATE and DELETE

```
UPDATE Appointment SET Status = 'Completed' WHERE Appointment_ID = 5007;

UPDATE Doctor SET Consultation_Fee = Consultation_Fee * 1.10
WHERE Specialization = 'Oral Surgeon';

DELETE FROM Appointment WHERE Appointment_ID = 5009;
```

The first command marks one appointment as completed, and the second increases the fee of the oral surgeon by 10 percent, so his fee becomes 1320. The third removes the cancelled appointment. After these commands, 11 appointments remain, of which 6 are completed. The outputs in Chapter 7 are based on the data after these changes.

6.2 Oracle and MySQL

Most SQL commands are the same in Oracle and MySQL, but a few things are different:

Work	Oracle	MySQL
Text data type	VARCHAR2(50)	VARCHAR(50)
Number data type	NUMBER(4)	INT
Text to date	TO_DATE('02-11-2026', 'DD-MM-YYYY')	STR_TO_DATE('02-11-2026', '%d-%m-%Y')
Replacing NULL	NVL(column, 0)	IFNULL(column, 0)
Limiting rows	WHERE ROWNUM <= 5	LIMIT 5

7. Sample Queries and Output

Some important queries of the project are given below with their outputs. The outputs are shown in the same form in which they appear when the queries are run in the terminal.

Query 1: Show the ID, name, phone and area of all patients.

```
SELECT Patient_ID, Patient_Name, Phone, Area FROM Patient;
```

Output:

PATIENT_ID	PATIENT_NAME	PHONE	AREA
1001	Rahim Uddin	01720111222	Uttara
1002	Fatema Begum	01820333444	Mirpur
1003	Karim Islam	01920555666	Dhanmondi
1004	Ayesha Siddika	01720777888	Uttara
1005	Tanvir Hasan	01620111999	Mirpur
1006	Nusrat Jahan	01520222111	Banani
1007	Sabbir Ahmed	01320333222	Uttara
1008	Sharmin Akter	01420444333	Dhanmondi

Query 2: Show each area only once.

```
SELECT DISTINCT Area FROM Patient;
```

Output:

AREA
Uttara
Mirpur
Dhanmondi
Banani

Query 3: Show the treatment fee with an extra X-ray charge of 500 taka (alias and calculated field).

```
SELECT Treatment_Name AS Treatment, Base_Fee AS "Fee (BDT)",
       Base_Fee + 500 AS "Fee With X-Ray"
FROM Treatment;
```

Output:

TREATMENT	FEE (BDT)	FEE WITH X-RAY
Scaling & Polishing	1500	2000
Root Canal	8000	8500
Tooth Filling	2500	3000
Braces Consultation	1000	1500
Teeth Whitening	6000	6500
Tooth Extraction	3000	3500

Query 4: Show the female patients who live in Uttara (WHERE with AND).

```
SELECT Patient_Name, Area, Gender FROM Patient
WHERE Area = 'Uttara' AND Gender = 'F';
```

Output:

PATIENT_NAME	AREA	GENDER
Ayesha Siddika	Uttara	F

Query 5: Show the treatments whose fee is between 2000 and 7000 taka.

```
SELECT Treatment_Name, Base_Fee FROM Treatment
WHERE Base_Fee BETWEEN 2000 AND 7000;
```

Output:

TREATMENT_NAME	BASE_FEE
Tooth Filling	2500
Teeth Whitening	6000
Tooth Extraction	3000

Query 6: Show the patients whose phone number starts with 017 (LIKE).

```
SELECT Patient_Name, Phone FROM Patient
WHERE Phone LIKE '017%';
```

Output:

PATIENT_NAME	PHONE
Rahim Uddin	01720111222
Ayesha Siddika	01720777888

Query 7: Show the patients who did not give an email address (IS NULL).

```
SELECT Patient_Name, Phone FROM Patient
WHERE Email IS NULL;
```

Output:

PATIENT_NAME	PHONE
Karim Islam	01920555666
Tanvir Hasan	01620111999
Sharmin Akter	01420444333

Query 8: Show the doctors in descending order of consultation fee (ORDER BY).

```
SELECT Doctor_Name, Consultation_Fee FROM Doctor
ORDER BY Consultation_Fee DESC;
```

Output:

DOCTOR_NAME	CONSULTATION_FEE
Dr. Ibrahim Khalil	1500
Dr. Shakhawat Hossain	1320
Dr. Rafea Oishee	1000
Dr. Saima Akter	800
Dr. A. S. Sarfuddin	800

The fee of Dr. Shakhawat Hossain is shown as 1320 because of the 10 percent increase made by the UPDATE command in Chapter 6.

Query 9: Find the total number of payments, total collection, and the average, smallest and largest payment.

```
SELECT COUNT(*) AS Total_Payments, SUM(Amount_Paid) AS Total_Collection,
       AVG(Amount_Paid) AS Average_Payment, MIN(Amount_Paid) AS Smallest,
       MAX(Amount_Paid) AS Largest
FROM Payment;
```

Output:

TOTAL_PAYMENTS	TOTAL_COLLECTION	AVERAGE_PAYMENT	SMALLEST	LARGEST
6	24100	4016.67	2300	7000

Query 10: Count the appointments of each doctor, keeping only the doctors who have more than one appointment (GROUP BY and HAVING).

```
SELECT Doctor_ID, COUNT(*) AS Total_Appointments FROM Appointment
GROUP BY Doctor_ID
HAVING COUNT(*) > 1
ORDER BY Doctor_ID;
```

Output:

DOCTOR_ID	TOTAL_APPOINTMENTS
101	4
103	2
104	2
105	2

Here WHERE and HAVING are different: WHERE selects rows before grouping, while HAVING selects groups after grouping.

Query 11: Show the patients who have at least one completed appointment (IN with subquery).

```
SELECT Patient_Name, Area FROM Patient
WHERE Patient_ID IN (SELECT Patient_ID FROM Appointment
                     WHERE Status = 'Completed');
```

Output:

PATIENT_NAME	AREA
Rahim Uddin	Uttara
Fatema Begum	Mirpur
Karim Islam	Dhanmondi
Ayesha Siddika	Uttara
Tanvir Hasan	Mirpur
Sabbir Ahmed	Uttara

Query 12: Prepare a billing summary showing the total bill, the amount paid and the balance due for each appointment (NVL).

```
SELECT a.Appointment_ID AS Appt, p.Patient_Name AS Patient,
       t.Base_Fee + d.Consultation_Fee AS Total_Bill,
       NVL(SUM(pay.Amount_Paid), 0) AS Paid,
       (t.Base_Fee + d.Consultation_Fee)
         - NVL(SUM(pay.Amount_Paid), 0) AS Balance_Due
FROM Appointment a
JOIN Patient p ON p.Patient_ID = a.Patient_ID
JOIN Doctor d ON d.Doctor_ID = a.Doctor_ID
JOIN Treatment t ON t.Treatment_ID = a.Treatment_ID
LEFT JOIN Payment pay ON pay.Appointment_ID = a.Appointment_ID
GROUP BY a.Appointment_ID, p.Patient_Name, t.Base_Fee, d.Consultation_Fee
ORDER BY a.Appointment_ID;
```

Output (first six appointments):

APPT	PATIENT	TOTAL_BILL	PAID	BALANCE_DUE
5001	Rahim Uddin	2300	2300	0
5002	Fatema Begum	7000	7000	0
5003	Karim Islam	9320	9000	320
5004	Ayesha Siddika	3300	3300	0
5005	Tanvir Hasan	2500	2500	0
5006	Nusrat Jahan	3800	0	3800

In this query NVL is used so that an appointment with no payment shows 0 as the paid amount instead of a blank. Appointment 5006 is still scheduled, so its full bill of 3800 taka is due.

8. Conclusion

In this project, a database for a dental clinic has been designed and implemented using Oracle SQL. The work started from the ER model of the clinic, and the model was then converted into six relational tables: Chamber, Doctor, Patient, Treatment, Appointment and Payment.

DDL commands were used to create the tables and to change their structure, and DML commands were used to insert, update and delete records. With SELECT queries, the stored data was searched and arranged in different ways. Filtering, sorting, aggregate functions, grouping with GROUP BY and HAVING, and subqueries were used to answer practical questions, such as the total collection of the clinic and the balance due from a patient.

The project has some limitations. It has no user interface, so all the work must be done by writing queries. It also does not store the patient's previous treatment history or medical reports. In the future, a simple application can be connected with this database, and more tables can be added for prescriptions and follow-up visits.

Working on this project has given a clear practical idea of how a database is designed from an ER diagram and how SQL is used to manage and search data in a systematic way.